```
    entanglement_blocks='cz',
    reps=2
)
Optimization
optimizer = COBYLA(maxiter=100)
Build VQC Model
sampler = Sampler()

vqc = VQC(
    sampler=sampler,
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=optimizer
)
vqc.fit(X_train, y_train)
<qiskit_machine_learning.algorithms.classifiers.vqc.VQC at
0x708a7a0a2e80>
accuracy = vqc.score(X_test, y_test)
print("VQC Accuracy:", accuracy)
```

Your implementation is structurally correct, but you should not assume this is “better than QSVM” or “inherently powerful.” A **Variational Quantum Classifier (VQC)** is highly sensitive to design choices (ansatz, optimizer, encoding). Right now, your setup is **minimal and likely under-optimized**.

---

1. What you built

You implemented a **Variational Quantum Classifier**.

Core idea:

Instead of computing a kernel (like QSVM), you:

- Encode data into a quantum circuit
- Add **trainable parameters**
- Optimize those parameters using classical optimization

Mathematical View

Your model is effectively learning:

$$f(x, \theta) = \langle \psi(x, \theta) | M | \psi(x, \theta) \rangle$$

- x → input data

- θ → trainable parameters
- M → measurement operator

👉 Output → classification score

---

2. Pipeline Breakdown

🔹 Dataset

```python
X = X      2
```

Critical issue:

- You reduced **30 → 2 features**

❗ This is a major bottleneck
👉 Model capacity > data representation

🔹 Feature Map

```python
ZZFeatureMap feature_dimension=2  reps=2
```

Role:

- Converts classical data → quantum state

👉 Adds:

- non-linearity
- entanglement

🔹 Ansatz (Trainable Circuit)

```python
TwoLocal
    num_qubits=2
    rotation_blocks='ry'
    entanglement_blocks='cz'
```

```
    reps=2
```

What this means:

- `ry` → rotation gates (parameters)
- `cz` → entanglement
- `reps=2` → depth of circuit

👉 This defines your **model capacity**

🔹 Optimizer

```python
COBYLA maxiter=100
```

Properties:

- Gradient-free
- Works well for noisy functions
- BUT:
  - slow convergence
  - may get stuck

🔹 Training

```python
vqc.    X_train  y_train
```

👉 Hybrid loop:

1. Quantum circuit runs
2. Output measured
3. Classical optimizer updates parameters

🔹 Evaluation

```python
accuracy = vqc.      X_test  y_test
```

---

3. What is actually happening internally

Step-by-step flow:

1. Input x
2. Apply feature map → $|\phi(x)\rangle$
3. Apply ansatz → $U(\theta)|\phi(x)\rangle$
4. Measure expectation
5. Convert to label

---

4. Key Insight (important)

👉 QSVM vs VQC difference:

QSVM	VQC
Fixed kernel	Learnable model
No training of quantum circuit	Train parameters
Convex optimization	Non-convex optimization

---

5. Major Limitations in Your Setup

⚠️ 1. Low Feature Dimension

- Using only 2 features
- You are **throwing away most signal**

👉 Likely hurting performance more than helping quantum part

⚠️ 2. Shallow Circuit

- `reps=2` is small
- Limited expressibility

⚠️ 3. Optimizer Choice

- COBYLA is:
  - robust
  - but inefficient

⚠️ 4. No Regularization / Validation

- No hyperparameter tuning
- No cross-validation

---

6. Debugging Workflow (very important)

If accuracy is poor:

Check 1: Feature bottleneck

Try:

```python
X = X      4
```

👉 But then:

- Need 4 qubits

Check 2: Ansatz depth

```python
reps=3 or 4
```

Check 3: Optimizer

Try:

- SPSA (better for noisy quantum)
- Adam (if simulator)

Check 4: Overfitting

- Compare train vs test accuracy

---

7. Why VQC can fail (core theory)

Barren Plateau Problem

👉 Gradients become ~0

- Happens in deep circuits
- Optimization stops

Expressibility vs Trainability tradeoff

- More layers → expressive
- But harder to train

---

8. Why your model may not outperform classical

You might assume:

> "Quantum model should be better"

❌ Not true here

Reasons:

- Small dataset
- Low dimension
- Simulator (no quantum advantage)
- Classical SVM is already strong

---

9. When VQC actually shines

VQC is useful when:

- Data is high-dimensional
- Complex feature interactions
- Quantum hardware advantage exists

---

10. One-line intuition

👉 VQC =
“Neural network, but implemented using quantum gates”

---

11. Comparison Summary

Model	Strength	Weakness
QSVM	Stable, convex	No learning
VQC	Learnable	Hard to train

---

12. What you should improve (practical)

Minimal improvement plan:

1. Increase features (2 → 4)
2. Increase ansatz depth
3. Change optimizer
4. Compare with classical baseline

---

13. Honest Evaluation

Your implementation is:

✔ Correct
✔ Good for demonstration
❗ Not optimized
❗ Not showing quantum advantage

---

14. If you want next-level work

I can help you build:

1. Production-grade VQC

- better ansatz design

- parameter initialization strategy

2. Research-level comparison

- VQC vs QSVM vs classical models
- statistical significance

3. Dataset selection (important for your profile)

- non-linear, quantum-friendly datasets

Before that, clarify:

Do you want to optimize:

1. **Accuracy**
2. **Theoretical understanding**
3. **Research/project-level implementation**

Your next step depends on this.